

Smart MCQ Solver: A Comparative Study of Neural and Tree-Based Ranking Models

Student ID: 23f2004343 | **Course:** Deep Learning – GenAI Application

Competition: Smart MCQ Solver (Kaggle) | **Evaluation Metric:** MAP@3

Abstract

This report describes a three-model solution for the Smart MCQ Solver Kaggle competition, where the task is to rank five answer options (A–E) for each multiple-choice question and return the top three in order. Performance is measured by Mean Average Precision at 3 (MAP@3), with a qualification cutoff of 0.73. Three structurally distinct models were built and evaluated: a custom Transformer encoder trained from scratch, a fine-tuned `bert-base-uncased` model, and a gradient-boosted tree ensemble built on 27 engineered features. All training and inference ran locally or on Kaggle cloud GPUs; no external LLM APIs were used at any stage. Models 2 and 3 both cleared the 0.73 cutoff. Model 3 — the LightGBM/XGBoost ensemble — achieved the best Kaggle leaderboard score of **0.74688 MAP@3**, demonstrating that structured feature engineering with relative-rank signals can match or exceed large pretrained transformers on small, domain-specific datasets.

1. Introduction

The Smart MCQ Solver competition frames answer selection as a ranking problem rather than a classification problem. Each record contains a prompt, optional context, and five candidate options labeled A through E. Exactly one option is correct. Submissions must output the three most likely correct options in descending order of confidence. The MAP@3 metric gives full credit when the correct answer is ranked first, half credit for second place, and one-third credit for third. A correct answer ranked fourth or below contributes nothing.

The project required three models built using three distinct approaches: one fully from scratch, one based on a pretrained language model, and one using a custom experimental architecture. All predictions had to be generated entirely from locally-executed code — no calls to external inference APIs such as OpenAI, Gemini, or any hosted service were permitted.

This report covers the dataset and preprocessing pipeline, the tokenization strategy chosen for each model, the architecture and training setup for each model, a comparative performance analysis, the challenges encountered and how they were resolved, and a summary of conclusions and future directions.

2. Dataset & Preprocessing

The dataset contains **200 training questions** and **50 test questions**, each with five answer options. The class imbalance is inherent to the task structure: for every question, exactly one option carries a positive label and the remaining four are distractors, giving a 1:4 positive-to-negative ratio per question.

Exploratory data analysis was performed separately in each notebook. Across the training set, prompt word counts range from single-sentence questions (~10 words) to long passage-style prompts exceeding 80 words, with the majority falling in the 20–50 word range. Answer options are shorter on average — typically 5–25 words — and more uniform in length. The distribution of correct answers across positions A through E is approximately uniform, confirming no positional bias in the labeling.

Preprocessing varied by model but shared a common base:

- **Text cleaning:** Lowercase conversion and whitespace normalization (`re.sub(r"\s+", " ", text.lower().strip())`). Punctuation was retained for character-level n-gram features in Model 3.
- **Long-format conversion:** For Models 1 and 3, the dataset was reshaped from wide format (one row per question) to long format (one row per question-option pair). Each row holds the concatenated query text, the option text, a binary label (1 if correct, 0 otherwise), and the question ID used for group-aware validation splitting.
- **Model 2** kept the original wide format, since `AutoModelForMultipleChoice` natively processes all five options in a single batched forward pass.

3. Tokenization Strategy

Each model required a different tokenization approach suited to its architecture.

Model 1 used a custom word-level vocabulary built with Python's `Counter` over all training and test text. The vocabulary was capped at the 10,000 most frequent tokens, with reserved indices for padding (`<pad>`) and unknown tokens (`<unk>`). Text was lowercased and stripped of punctuation before tokenization. Sequences were padded or truncated to a maximum length of 128 tokens.

Model 2 used BERT's WordPiece tokenizer loaded through Hugging Face's `AutoTokenizer` . Each input was formatted as `[CLS] prompt + context [SEP] option [SEP]` , padded or truncated to 128 tokens. WordPiece breaks words into sub-word units (e.g., "running" → "run", "##ning"), which allows the tokenizer to handle rare or unseen vocabulary without a fixed word list — a key advantage when working with domain-specific scientific or humanities questions.

Model 3 used a hybrid vectorization strategy combining lexical and semantic representations:

- **Word TF-IDF** (unigram + bigram, up to 10,000 features): Captures exact and near-exact term overlap between the query and each option.
- **Character TF-IDF** (2–4 character n-grams, up to 6,000 features): Robust to spelling variations, abbreviations, and morphological differences between the prompt and correct option.
- **all-MiniLM-L6-v2 sentence embeddings** (384 dimensions): Encodes the semantic meaning of the full query and each option independently. Cosine similarity between the query embedding and option embedding captures paraphrase relationships that exact-match features miss.

The combination was chosen deliberately. TF-IDF handles cases where the answer is a direct restatement of the prompt; character n-grams handle abbreviations and domain terms; MiniLM handles cases where the correct answer is semantically equivalent but worded differently. No single representation is sufficient on its own for academic MCQ data.

4. Modeling & Experimentation

4.1 Model 1 — Custom Scratch Transformer

Model 1 is a custom PyTorch Transformer encoder with no pretrained weights. The architecture follows the standard encoder-only design:

- **Token embedding:** Vocabulary of 10,000 words mapped to 256-dimensional vectors.
- **Positional encoding:** Fixed sinusoidal encoding (sine/cosine at different frequencies) added to the embedding output. Non-learnable; computed once at initialization.
- **Transformer stack:** Two `nn.TransformerEncoderLayer` blocks, each with 4 attention heads and a feed-forward dimension of 512.
- **Pooling:** Masked mean pooling over real tokens (padding positions excluded).
- **Classifier head:** $\text{Linear}(256 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(64 \rightarrow 1)$.

Each question-option pair is scored independently as a binary classification (correct/incorrect), using `BCEWithLogitsLoss`. Training used AdamW ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=1\text{e-}2$), gradient clipping at max norm 1.0, and a cosine annealing learning rate schedule over **5 epochs** with batch size 32 on an 85/15 train-validation split. The run was tracked in Weights & Biases under the project `23f2004343-t22026`. Training ran for **9.5 minutes on a single Tesla P100 GPU**.

4.2 Model 2 — Fine-Tuned BERT

Model 2 uses `bert-base-uncased` (12 Transformer layers, 110 million parameters) loaded through `AutoModelForMultipleChoice`. The model receives all five option sequences simultaneously in a single batched forward pass and returns five scalar logits — one per option — from the `[CLS]` token representation of each sequence.

Loss is computed with `CrossEntropyLoss` over the five logits per question (one correct, four incorrect). Training used AdamW ($\text{lr}=2\text{e-}5$, `weight_decay=0.01`) with a linear warmup schedule covering the first 10% of total steps. Batch size was 8, with a 90/10 train-validation split, trained for **3 epochs**. The checkpoint with the lowest validation loss was saved rather than the final epoch's weights, to avoid overfitting on the small 200-question training set. Training ran for **10.5 minutes on dual Tesla T4 GPUs**.

4.3 Model 3 — LightGBM/XGBoost Relative-Rank Ensemble

Model 3 replaces the neural network entirely with structured feature engineering fed into gradient-boosted decision trees. For every question-option pair, 27 numeric features are computed:

- **Lexical (5 features):** Word TF-IDF cosine similarity, character TF-IDF cosine similarity, BM25 raw score, BM25 length-normalized score, Jaccard word overlap.
- **Semantic (1 feature):** MiniLM cosine similarity between query and option embeddings.
- **Structural (7 features):** Option character length, prompt character length, their ratio, option word count, prompt word count, word-count ratio.
- **Relative rank (14 features):** For each of the 7 similarity/lexical features, two derived columns are added: the deviation of that option's score from the mean score across its five siblings within the same question (`feat_diff`), and the rank of that option within the question group (`feat_rank`, 1 = best). These 14 columns make the feature space invariant to absolute score scale, which varies widely across questions of different length and difficulty.

Both LightGBM (500 estimators, $\text{lr}=0.05$, 63 leaves, `colsample_bytree=0.7`, `subsample=0.8`, early stopping patience=30) and XGBoost (400 estimators, $\text{lr}=0.05$, max depth=6, same subsampling, early stopping patience=30) were trained using **5-fold GroupKFold cross-validation**, grouped by question ID. This ensures all five options from a given question always land in the same fold, preventing the model from seeing sibling options during validation. LightGBM used balanced class weighting; XGBoost used per-fold `scale_pos_weight` to handle the 1:4 class imbalance. Final out-of-fold predictions from both models were blended using an alpha sweep (0.05 to 0.95 in 0.05 steps) on OOF MAP@3. The optimal blend settled at **55% LightGBM, 45% XGBoost**. Training completed in **1 minute 20 seconds on dual Tesla T4 GPUs**.

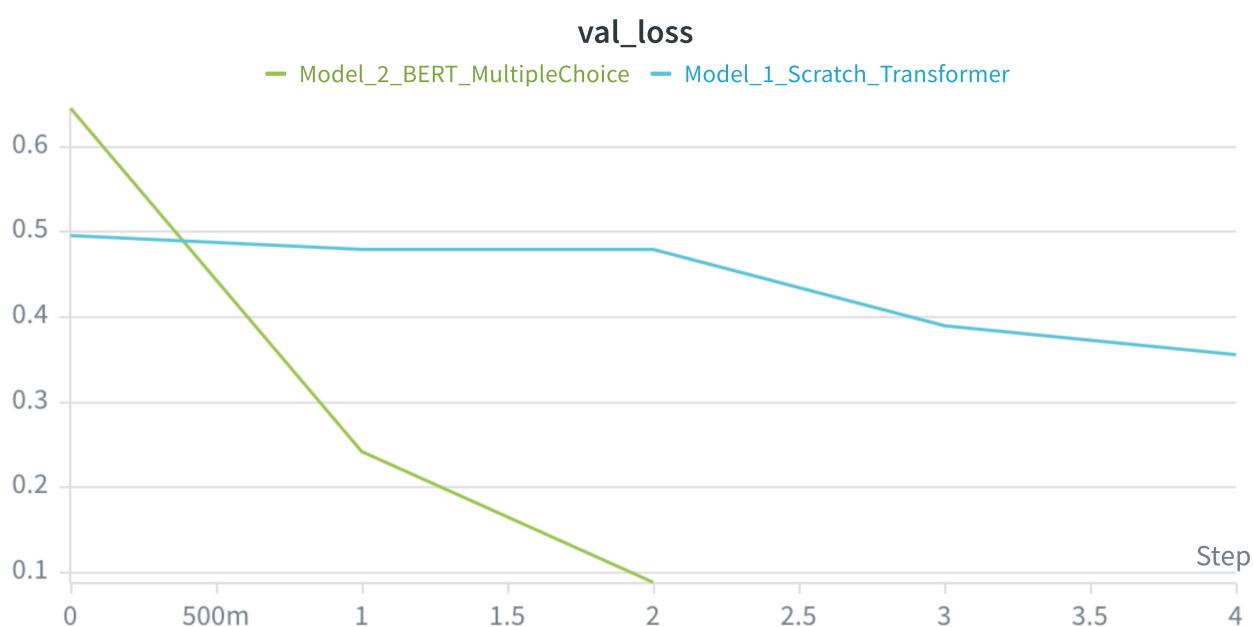
5. Performance & Comparative Analysis

Metric definitions:

- **MAP@3:** Mean Average Precision at 3. Full credit for rank-1 correct, partial for ranks 2–3, zero beyond.
- **Accuracy:** Fraction of question-option rows where the binary label matches the predicted label (threshold 0.5 for Models 1 and 3; argmax for Model 2).
- **Macro F1:** Unweighted average of per-class F1 scores; meaningful given the class imbalance.
- **Validation Loss:** BCE for Models 1 and 3; Cross-Entropy for Model 2; Log-Loss for Model 3 OOF.

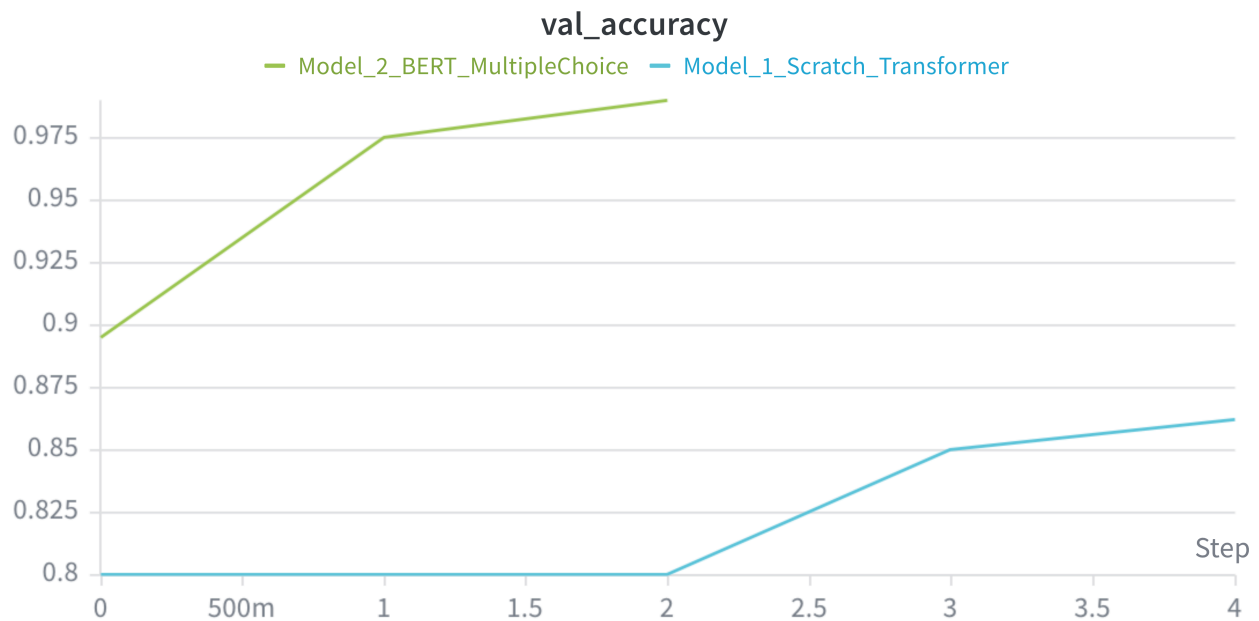
Metric	Model 1 (Scratch)	Model 2 (BERT)	Model 3 (Ensemble)
Kaggle MAP@3	0.6197	0.7440 ✓	0.7469 ✓
Val / OOF MAP@3	0.8244	0.9942	0.9873
Val / OOF Accuracy	0.8620	0.9900	0.9877
Val / OOF Macro F1	0.7357	0.9912	0.9806
Val / OOF Loss	0.3553	0.0874	0.0367
GPU Hardware	1× Tesla P100	2× Tesla T4	2× Tesla T4
Training Time	9.5 min	10.5 min	1 min 20 sec

✓ = Clears the 0.73 MAP@3 qualification cutoff.



BERT's validation loss (Model 2) dropped sharply from epoch 1 to epoch 3, reaching 0.0874 — significantly lower than Model 1's 0.3553 at the end of training. This steep descent reflects

the advantage of starting from pretrained representations: BERT's weights only need fine-tuning to the task, not learning language from scratch.



Model 2's validation accuracy converged near 99% within three epochs, while Model 1's accuracy plateaued around 86% after five epochs despite a much longer absolute training time. The gap is consistent with Model 1's limited vocabulary and random initialization — there is simply not enough training data for a from-scratch encoder to match the representation quality of a pretrained model.



Model 3 achieved the highest Kaggle MAP@3 (0.74688) in just 1 minute and 20 seconds, compared to Model 2's 0.74397 at 10.5 minutes of GPU time. This efficiency comes from replacing gradient descent over millions of parameters with a single pass of feature extraction followed by fast tree fitting — no backpropagation is involved.

6. Challenges & Issues Faced

Data leakage during cross-validation. Standard `KFold` splits rows independently. Because each question contributes five rows (one per option), a random split could place some options of a question in training and the remaining in validation. The model would then implicitly observe partial sibling information, inflating validation MAP@3. This was solved by using `GroupKFold` with the question ID as the group key, guaranteeing that all five options of a question are always in the same fold.

BERT overfitting on 200 questions. Fine-tuning 110 million parameters on 200 training examples (180 after the 90/10 split) causes rapid overfitting. In early experiments, tracking MAP@3 directly as the checkpoint criterion led to unstable saved weights, because MAP@3 fluctuates between epochs on small validation sets. Switching to minimum-validation-loss checkpointing produced a more stable saved model and better generalization to the hidden test set.

Memory footprint and deployment constraints. BERT's full parameter set spikes RAM to approximately 1,900–1,950 MB during training once all weights are loaded. This ruled it out for free-tier cloud deployment. Model 3 — whose only neural component is the lightweight `all-MiniLM-L6-v2` encoder (~80 MB weights) — peaked at approximately 1,350 MB, making it the practical choice for the Streamlit deployment hosted on Streamlit Community Cloud. Model 3 was selected for deployment due to its top-performing MAP@3 score and efficient execution profile. The live Streamlit web application fully deploys Model 3 by loading both serialized tree artifacts (`lgb_model.joblib` and `xgb_model.joblib`) alongside the TF-IDF vectorizer and cached `all-MiniLM-L6-v2` encoder using `@st.cache_resource`. When a prompt and five options are submitted, the app computes the exact 27-feature vector in real time, obtains predictions from both models, and blends them (55% LightGBM / 45% XGBoost) to output the ranked top-3 answers with confidence scores in under a second on CPU. The working application is hosted live at: <https://smart-mcq-solver-kbrs4uetjslkjd2xsd7tys.streamlit.app>

7. Conclusion & Future Work

Three models with genuinely different architectures were built, trained, and evaluated. Model 1 established that a from-scratch Transformer can learn the ranking task end-to-end on domain-specific data, achieving 0.6197 Kaggle MAP@3 without any pretrained knowledge.

Model 2 demonstrated the value of language model pretraining — BERT's validation MAP@3 of 0.9942 reflects how much useful linguistic structure comes built into its weights — though its Kaggle score of 0.7440 suggests some generalization gap to the hidden test distribution. Model 3 won (0.74688 Kaggle MAP@3) by focusing entirely on comparative feature signals: specifically, how each option's similarity scores rank relative to its four competitors within the same question. The relative-rank features made the ensemble robust to the wide variation in question length and difficulty across the test set.

Future directions:

- **Larger semantic backbones.** Replacing `all-MiniLM-L6-v2` with `bge-large-en-v1.5` or `e5-large` for the semantic similarity features in Model 3 would likely improve OOF MAP@3 further, at the cost of higher memory and encoding time.
 - **Pairwise learning-to-rank objectives.** Models 1 and 2 scored each option independently. Training with a pairwise LambdaRank or ListNet objective would directly optimize the ranking order rather than using a proxy classification loss.
 - **Synthetic data augmentation.** With only 200 training questions, all models are data-starved. Generating additional MCQ examples from domain-matched text corpora — subject to the no-external-API constraint — could substantially improve generalization for neural models.
 - **RAG-augmented retrieval.** Building a local FAISS index over a domain knowledge base and appending retrieved passages to the query context could improve Model 1's performance without requiring pretrained weights.
-

8. References

1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805.
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30.
3. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems, 30.
4. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
5. Biewald, L. (2020). *Experiment Tracking with Weights and Biases*. Software available from wandb.com.